Classical and Quantum Approaches to Factorization

Carson Ihrke Will Geiger

I. Introduction

When data is transferred over the internet, many times it is of extreme importance that the data remains secure i.e. maintains integrity, authenticity and confidentiality. To accomplish this, symmetric key algorithms are implemented to encrypt so that data can be transferred safely. The downside to these algorithms is that the key for decryption must also be shared. Asymmetric algorithms such as RSA were developed so that keys can be shared safely allowing safe data transportation end to end. As a result, RSA is a backbone of many protocols defining secure network communication such as TLS and SSH [1].

These types of cryptographic algorithms base their security on a variety of different, computationally complex mathematical problems. Problems such as prime factorization and discrete logarithms allow easy and straightforward computation given that the user knows the key while remaining difficult to reconstruct the key given known information. For a long time these problems remained a sturdy foundation for asymmetric algorithms, allowing much of modern cryptography to be built upon algorithms relying on this complexity.

Following the formation of the field of quantum computing, these security foundations were challenged. Many mathematicians, physicists and computer scientists have jumped into this field leveraging quantum mechanics with computing to produce algorithms that solve tasks exponentially faster than classical computers. Peter Shor was an early researcher in this field and developed quantum algorithms that can factor large numbers and calculate discrete logarithms at speeds in which the security of RSA breaks down [2].

Although not yet ready for real world usage, quantum computers provide an urgency for expanding the mathematical foundation of cryptographic algorithms and developing even tougher encryption schemes to break [3]. This project is aimed to explore the advantage of quantum computing in this way, comparing results between classical and quantum brute force decryption of RSA.

The following walks through necessary background information for our project followed by methods used to analyze current classical and quantum factorization methods to break RSA as well as results obtained from our experimentation. Everything is wrapped up with a brief section providing possible future work and our conclusive thoughts.

II. Background

RSA bases its security on the difficulty of factoring some number N made by the product of two large primes p and q. Given the product N, an individual (A) who needs to collect a secure key calculates a public and private key pair e and d.

The public key e is selected subject to two constraints. Because N is the product of primes we can use Euler's totient function $\phi(N)$ to calculate the number of coprimes to N. The totient function for N is defined as $\phi(N) = (p-1)(q-1)$ [4]. Given this value for $\phi(N)$, e is selected from the range $1 < e < \phi(N)$ where $\gcd(e, \phi(N)) = 1$. The public key pair that is distributed for encryption becomes (N, e). The value d is then calculated as the modular multiplicative inverse of e within the modular group $\phi(N)$.

The individual (A) can openly share the public key e and N, allowing other individuals (B) to encrypt data and send it back to (A). The private key held by (A) undoes the encryption performed by (B) thus having safely transported arbitrary data across both parties.

The process that RSA uses to ensure secure key sharing across two parties makes it simple for (B) to encrypt the secure key given the public key, however for some individual (C) or even (B) who does not know the private key, decryption becomes very difficult. Because d is calculated as a result of the selected values p and q the main objective for a third party to decrypt without knowledge of d is finding the factors of N [5].

Because the value d needed to decrypt an RSA encrypted message ultimately relies on the values of N and $\phi(N)$ it is necessary to find the factors p and q of N. On a classical computer this can be solved using various methods of guessing factors of N until they are found. As of now there are no known algorithms for guessing these factors in polynomial time which is why it is extremely computationally expensive to brute force RSA. This is why RSA is a gold standard for key exchanges for symmetric algorithm use.

On a quantum computer this is not the case. The crucial step in classically searching for factors of N uses the fact that for any two numbers a and b who share no factors, when a is raised to a power n, for some power a^n becomes some multiple of b plus one. Applying this to some guess g and N we find a formula describing factors of N, $(g^{\frac{n}{2}}+1)(g^{\frac{n}{2}}-1) = m \cdot N$.

In a classical setting this calculation might have to be done for an unmanageable number of guesses g and powers n, but in a quantum computer this large number of calculations for guesses of g can be done simultaneously. This process can be thought of as searching for the period of g in N because after a certain point the powers of g exhibit a periodic behavior within a modulus of N.

In such a quantum setting, laws of quantum mechanics

take effect and superposition and entanglement along with interference can be leveraged to quickly find the period. The process of finding factors p and q is done almost entirely classically; however to find the correct period for a guess g efficiently, we need all possible guesses g to be put in superposition.

The fundamental unit of data in a quantum computer, the qubit, is represented as a state vector within a complex vector space. Under the principle of quantum superposition, the overall state of the quantum system is described not by a single value, but as a linear combination of its computational basis states summed together simultaneously. This allows the state of our system to represent every power we need to compute for a guess g at the same time.

Quantum entanglement dictates that an entangled system cannot be described as a collection of independent individual states, but rather as one global state. Consequently, upon measurement of one qubit within the entangled system, the overall global state collapses instantly, simultaneously determining the measurement outcomes of all other qubits entangled with it. By entangling a qubit representing the modular arithmetic of $g^k \mod (N)$ with a qubit storing k we can represent all powers of k and the resulting modular arithmetic in one global superposition [6].

Within this superposition, incorrect periods destructively interfere while the correct period is revealed, requiring these behaviors of quantum mechanics. Using an inverse Quantum Fourier transform we can shift our system from the frequency basis to the computational basis and measure a candidate solution for finding our factors [2].

III. Methods

To achieve the desired goal of comparing classical and quantum algorithms designed to factor the product of two large primes we use classical methods of computation. To evaluate the advantage of Shor's algorithm we use a classical quantum simulator. It is important to recognize that this simulation of a quantum computer still uses classical resources and as such will exhibit classical computing power. The simulation serves to show on a small scale the computational advantage a quantum computer holds over its classical counterpart. This analysis is still of importance due to the lack of quantum hardware necessary to handle computation.

To handle development and execution of algorithms we use Google Colab as the development environment to organize components efficiently. The language used in development is Python. Python allows for readable, easy to follow algorithms as well as support for classical quantum simulation. The python SDK, Qiskit, offered by IBM allows developers to build quantum circuits equivalent to the architecture of a quantum processor. From here we can manipulate simulated qubits on a simulated processor, effectively demonstrating Shor's algorithm.

Additionally, modular arithmetic for RSA requires certain precision and operation so we use the Python SymPy package to handle these calculations. The Matplotlib package is used for visualization of benchmarks and results.

Beginning this project, we need a functioning RSA encryption scheme. The development of this requires generating random primes and using their product to generate our public and private keys. This was successfully implemented using components of the SymPy package for careful arithmetic. We then developed an algorithm performing the computations necessary for RSA encryption/decryption using the generated keys. This requires handling the translation of string data into integer data for calculation both when encrypting and decrypting. The overall development process of this was straightforward.

With an RSA environment established, we next moved on to classical methods of factoring for the purpose of brute-force cryptanalysis. To establish our baseline performance, we began developing the algorithm to factor the modulus N. Our experimental setup for this involves feeding the classical algorithm an increasingly large modulus, starting at small 8 bit keys and scaling up toward a target of 40 bit keys. For the increasing key size we record the CPU execution time. This benchmarking allows the mathematical mapping of the exponential execution curve that forces classical hardware to hit a computational wall.

Three purely classical methods of factoring were explored. A linear search algorithm was implemented, linearly testing all possible factors of N up to $\sqrt{N}$. Additionally, the classical factoring method that Shor's algorithm enhances, a period search, was implemented for further value to our analysis. Fermat's Factorization was implemented, however benchmarking of this algorithm was omitted because of the precise conditions required for its use. In practice, a modulus N that can be factored by Fermat's method is the result of poorly selected p and q values [7].

Along with developing classical implementations, using Qiskit, we constructed a fully generalized simulation of Shor's Algorithm that scales its register size based on the target modulus N. To get an understanding of working with Qiskit, we followed a Shor's algorithm example solution for a specific target N [8] and generalized the process for any N.

This simulation dynamically configures the quantum workspace by initializing a control register alongside a target register in order to ensure adequate computational resources. We initialize the system by shifting the target register into the $|1\rangle$ state via a Pauli-X gate and applying Hadamard gates across the control register to induce an equal superposition of all possible states.

We then used NumPy to build a permutation matrix (see Figure 1A in appendices) for the transformation $|x\rangle \rightarrow |(a^{2^k} \cdot x) \pmod N\rangle$, and applied Qiskit's UnitaryGate operation. After running the control qubits through this modular multiplication sequence, the register passes through an Inverse Quantum Fourier Transform to shift the phase data back into measurable computational basis states.

To validate our generalized architecture, we executed our script using the simulation backend, setting our target modulus

to $N = 33$ with a base guess of $a = 2$. Running the circuit for 50 shots successfully returned the correct period, allowing our post processing script to extract the final prime factors.

Our classical post-processing script successfully converted the resulting period allowing continued fractions to isolate the true period, and executed the final greatest common divisor calculation. The system successfully isolated the valid period and correctly extracted the true prime factors 3 and 11, proving that our generalized code functions.

To analyze the results of our algorithms a helper algorithm was developed. This environment set up a scaling suite of different bit values of N. Using these incrementing bit sizes of N, the helper ran the various implemented factoring algorithms to find factors of N, keeping track of execution times for classical methods and tracking the various measurements made through quantum trials.

IV. Results

The results obtained from benchmarking our various algorithms supported the overall computational wall classical factoring experiences. As bit size for N increased, the computational time to factor N remained relatively low, but as soon as a specific range was met, execution times spiked rapidly.

Figure 1. visualizes the execution time for linearly searching factors of N. Our personal laptops were able to efficiently compute factors up until N exceeded 40 bits. Beyond 40 bits is where we begin to see the computational wall factoring exhibits.

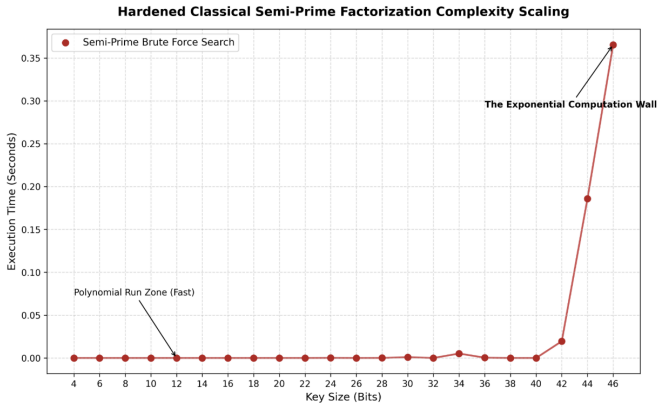

Figure 1: Computational time to linearly search factors of N.

Interestingly through trial and error generating primes, we have found that the computational power required to factor greatly depends on well chosen values of p and q. If a value p or q are chosen too small, a linear search can quickly solve for both. If p and q are too close together Fermat's factorization can efficiently compute the factors.

The analysis (Figure 2.) for our classical period finding algorithm was similar; however, a personal computer experiences trouble more rapidly when computing factors of N. Our algorithm was only able to factor up to a bit size of 28, where beyond 28 bits the algorithm would halt. Although this may be a concise method for guessing factors, the computational overhead required scales much more rapidly than that of linear search.

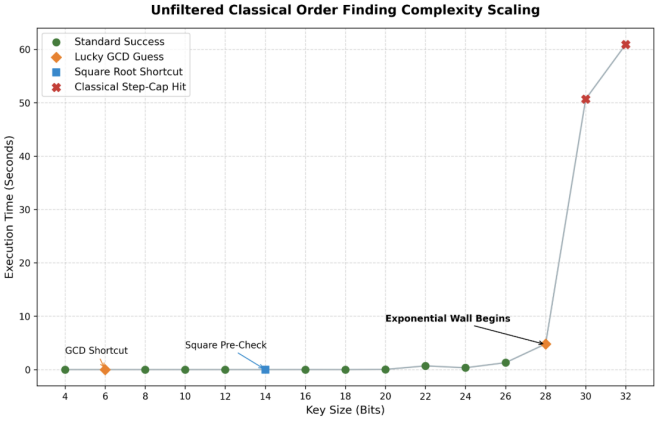

Figure 2: Computational time to find period within N.

As previously mentioned, the simulation of Shor's algorithm required classical computational resources. For this reason, computational overhead was extreme, providing no feasible method of analysis related to the process of our classical algorithm analysis. The simulation mostly served to explore how computational resources scale as input size scales.

To perform Shor's algorithm on actual quantum hardware, each gate required for computation such as the unitary gate built for modular arithmetic is hardwired. A system that can factor huge composite values N requires a scaled number as qubits as implemented in the simulation. Because Shor's algorithm exhibits a scaling of hardware rather than memory it allows for much more efficient computation.

Although within a classical simulation, we have successfully shown how hardware rather than memory scales and the quantum principles that allow for speedy computation within this problem. As shown in Figure 2A, the quantum system is manipulated in a way that upon measurement the probability of a useful phase being measured is amplified and allows for extreme efficiency in computing factors of N.

V. Future Work

One huge hurdle quantum computing faces is overcoming sources of noise that a quantum computing environment faces [6]. The Qiskit simulation we built in this project does not introduce noise that might be found in practice. To further explore both the simulation and how these algorithms operate at scale in real hardware, the introduction of noise to the system provides a great foundation to bridge the gap between simulation and practice.

Additionally within our classical factoring algorithms there are many edge case situations that can be computed extremely quickly. For example, if a number N is a square i.e. $p = q$, this can be checked for very fast. This check was included in the period finding algorithm but could be expanded to our linear search algorithm. To continue future work we could explore these possible edge cases and handle them quickly within our algorithms possibly boosting computational times.

To provide a much more thorough analysis regarding the security of RSA and the effect quantum computers can possibly have on the security reduction, an exploration of post quantum encryption algorithms could have been explored. NIST provides standards for such schemes based on shortest vector paths within multi dimensional lattices, hashing etc. [3]. Exploring such algorithms would add relevancy to recent advances in cryptography and cybersecurity.

VI. Conclusion

To conclude this project, we have successfully shown why RSA is safe from classical means of factoring while also exploring what will possibly be the downfall of RSA security. While modern public-key infrastructures rely heavily on the classical toughness of the prime factorization problem, the turn over to quantum architectures can fundamentally rewrite these security guarantees.

To map the operational boundaries of this computational problem, this paper successfully designed, executed, and analyzed two distinct factorization methodologies. On classical architectures, a benchmarking suite was deployed, demonstrating a severe exponential runtime scaling behavior as key lengths advanced. Conversely, a generalized quantum circuit framework was built using the IBM Qiskit SDK, utilizing an array of Hadamard gates to instantiate global superpositions and exploiting controlled modular multiplication layers to entangle exponent states with their corresponding remainders. By utilizing an inverse quantum Fourier transform, the quantum simulation successfully translated abstract phase periodicity into constructive and destructive wave interference patterns, collapsing the wave function cleanly onto correct candidate solutions.

Although Quantum computing remains restricted by current lack of decoherence mitigation, it poses significant risk to computational problems designed for classical computers. The time to develop quantum proof solutions is now and will continue to grow exceedingly more important as time moves on.

VII. Appendices

```
"""
constructs a controlled unitary matrix performing: |x> -> |(a_power * x) % N>
"""
def controlled_modular_multiplication(num_qubits, N, a_power):

    # initialize an empty operator matrix matching the size of the target register
    matrix = np.zeros((2**num_qubits, 2**num_qubits))

    # define permutation mappings for valid inputs under the modulus
    for x in range(2**num_qubits):
        if x < N:
            matrix[(a_power * x) % N, x] = 1
        else:
            matrix[x, x] = 1  # identity mapping for out-of-bounds states

    # convert the raw mathematical matrix into a qiskit standard unitary gate
    gate = UnitaryGate(matrix)
    gate.name = f"mod_{a_power}"

    # return the gate modified to accept a control qubit constraint
    return gate.control()
```

Figure 1A: Unitary gate wrapping for modular arithmetic mapping.

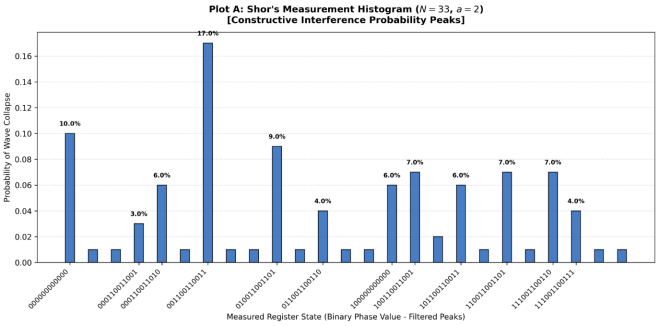

Figure 2A: Probability distribution of measurement collapsing to a given state.

VIII. References

[1] Mezher, Ahmed. (2018). Enhanced RSA Cryptosystem based on Multiplicity of Public and Private Keys. International Journal of Electrical and Computer Engineering (IJECE). 8. 3949. 10.11591/ijece.v8i5.pp3949-3953.

[2] Shor, P. W. (1997). Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer. SIAM Journal on Computing, 26(5), 1484–1509. https://doi.org/10.1137/s0097539795293172

[3] National Institute of Standards and Technology (NIST). 2025. "What Is Post-Quantum Cryptography?" Last modified May 16, 2025. https://www.nist.gov/cybersecurity-and-privacy/what-post-quantum-cryptography.

[4] Weisstein, Eric W. "Totient Function." From MathWorld–A Wolfram Web Resource. Accessed June 7, 2026. https://mathworld.wolfram.com/TotientFunction.html.

[5] GeeksforGeeks. 2026. "Difference Between Symmetric and Asymmetric Key Encryption." Last modified April 29, 2026. https://www.geeksforgeeks.org/computer-networks/difference-between-symmetric-and-asymmetric-key-encryption/.

[6] Krantz, Philip, Morten Kjaergaard, Feidhlim Yan, Orlando T. Orlando, Simon Gustavsson, and William D. Oliver. 2019. "A Quantum Engineer's Guide to Superconducting Qubits." Applied Physics Reviews 6 (2): 021318. https://doi.org/10.1063/1.5089550.

[7] GeeksforGeeks. 2024. "Fermat's Factorization Method." Last modified May 12, 2024. https://www.geeksforgeeks.org/dsa/fermats-factorization-method/.

[8] IBM Quantum. n.d. "Shor's Algorithm." IBM Quantum Documentation. Accessed June 8, 2026. https://quantum.cloud.ibm.com/docs/en/tutorials/shors-algorithm.